
Le langage procédural PL/pgSQL-Triggers

Dans tout l'énoncé, on suppose que le travail se fait avec le système d'information de la société Distribill (*i.e.* la base de données construite lors du TP2). Le schéma de la base de données est nommé `td4r206`.

Exercice 1 Un premier déclencheur (trigger)

Au sein de la société Distribill, il y a plusieurs secrétaires qui utilisent chacune leurs conventions de saisies des noms propres (première lettre en majuscule uniquement, tout en majuscules, voire tout en minuscules). Le comptable, pour des raisons de lisibilité, voudrait que les noms des employés soient tous enregistrés en majuscules, même si les entrées de l'utilisateur ne garantissent pas cette condition.

Vous allez réaliser un trigger qui assure cette fonctionnalité.

1. Quel est le type du trigger ? (ligne ou commande)
2. Sur quel(s) événement(s) se déclenche-t'il ?
3. Quand se déclenche-t'il ?
4. Implantez ce trigger (Vous devez programmer une fonction et un trigger qui l'exécutera, en réalité)
5. Vérifier la prise en compte de la contrainte en modifiant le nom de 'LEROY' en 'Dupont'.

La fonction de conversion d'une chaîne en majuscules est : `upper(chaine)`.

Exercice 2 Un déclencheur pour assurer la mise-à-jour du salaire des directeurs.rices

Créez un trigger qui se déclenche après chaque mise à jour de la colonne "poste" de la table `_employe`. Le trigger doit vérifier si le nouveau poste est éligible à une augmentation de salaire (les directeurs.rices sont les seules à être éligibles à cette augmentation). Si c'est le cas, le trigger doit mettre à jour le salaire de l'employé en ajoutant un pourcentage prédéfini (par exemple, 5%) à son salaire actuel.

1. Quel est le type du trigger ? (ligne ou commande)
2. Sur quel(s) événement(s) se déclenche-t'il ?
3. Quand se déclenche-t'il ?
4. Implantez ce trigger
5. Vérifier la prise en compte de la contrainte en modifiant le poste de 'DUBOIS' en 'Directeur'.

La fonction de mise-à-jour du salaire est : `mise_a_jour_sal()`.

Exercice 3 Un déclencheur pour assurer la cohérence des données à la création

On souhaiterait pouvoir écrire l'expression suivante :

```
insert into td4r206._employe (matr,nome,poste,sal,nodept,sup)
values (1234,'Dupont','TECHNICIEN',20000.00,20,7788);
```

On remarque que toutes les informations concernant l'employé ne sont pas présentes. Comme on essaie d'éviter les valeurs d'attributs à NULL, on supposera qu'à l'embauche :

- La valeur de la commission est nulle (égale à zéro).
- La date d'embauche est la valeur courante de la date.
- Le supérieur hiérarchique du nouvel embauché est affecté au même département que celui-ci sauf s'il s'agit d'un directeur. Dans ce cas le supérieur hiérarchique est obligatoirement le président.
- Pour la cohérence des données, un directeur ne peut être créé s'il existe déjà un directeur pour le département spécifié.

1. Proposez la fonction trigger et le déclencheur l'utilisant permettant de réaliser une insertion dans ces conditions.

Exercice 4 Un déclencheur pour assurer la cohérence des données à la suppression

Au moment de la suppression d'un employé, on souhaite également que les données restent cohérentes. On voudrait faire la suppression d'un employé :

- Seulement s'il ne s'agit pas d'un Directeur.
 - Si la suppression est possible, ses subalternes deviennent subalternes de son supérieur direct.
1. Modifiez le trigger précédent et sa fonction associée de manière à ce qu'il réagisse à l'ajout et à la suppression.
 2. Soit la table `_statistiques` à créer à l'aide du script SQL suivant :

```
CREATE TABLE td4r206._statistiques (  
  typemaj varchar(20) NOT NULL,  
  nbmaj integer NOT NULL,  
  timestampmodif timestamp,  
  constraint _statistiques_PK primary key (typemaj));
```

```
INSERT INTO td4r206._statistiques(typemaj,nbmaj, timestampmodif) VALUES('INSERT', 0, NULL);  
INSERT INTO td4r206._statistiques(typemaj,nbmaj, timestampmodif) VALUES('DELETE', 0, NULL);  
INSERT INTO td4r206._statistiques(typemaj,nbmaj, timestampmodif) VALUES('UPDATE', 0, NULL);
```

Définir un déclencheur après insertion ou suppression dans la table `_employe` qui met à jour automatiquement la table `_statistiques`. Tester son fonctionnement en effectuant quelques modifications dans la table `_employe`.

3. Tester l'effet des clauses `FOR EACH ROW` et `FOR EACH STATEMENT` sur le comportement du déclencheur en utilisant une requête `INSERT` qui ajoute plusieurs n-uplets.